Syrian Arab Republic

Ministry of Higher Education

Syrian Virtual University

Bachelor of IT Program BAIT

MS SQL Server Administration IIS404

Chapter eight

Auditing & Change Tracking in MS SQL Server

Eng. Ayham Mohammad

Syrian Virtual University SVU – 2020

Contents

1. Auditing with SQL Server and Azure SQL Database

- عملية ال audit هي عملية تتبع و حفظ ال Events في ال DB
- فس نسخة ال 2016 أصبح متوفّر ال Auditing في كل النسخ من ال SQL Server, حيث قبلها كان متوفر فقط في نسخة ال Enterprise

Auditing is the act of tracking and recording events that occur in the Database Engine. Since SQL Server 2016 Service Pack 1, the Audit feature is available in all editions, as well as in Azure SQL Database.

1.1. SQL Server Audit

- *يعتمد هذا ال Audit على إستخدام ال Extended Events ليعطينا معلومات على مستوى ال Instance وال DB و يضعها ضمن file (لا يقوم بتسجيلهم بشكل متزامن كي لا يؤثر على الأداء)*
-

There is a lot going on in the Database Engine. SQL Server Audit uses extended events to give you the ability to track and record those actions at both the instance and database level.

Audits are logged to event logs or audit files. An event is initiated and logged every time the audit action is encountered, but for performance reasons, the audit target is written to asynchronously.

The permissions required for SQL Server auditing are complex and varied, owing to the different requirements for reading from and writing to the Windows Event Log, the file system, and SQL Server itself.

1.1.1. Requirements for creating an audit

- المتطلبات لتفعيل ال Audit:

ملاحظة: لعمل ال **Audit** يجب على الأقل تفعيل خدمة ال **Audit** بالإضافة لتفعيل هذه الخدمة على ال **Server** أو ال **DB** حيث ليس إجباري تفعيلها على كلاهما و ليس إجباري تحديد ال **Target** (العمليات المراد مراقبتها – حيث سيتم مراقبة الإفتراضية)

To keep track of events (called actions), you need to define a collection, or *audit*. The actions you want to track are collected according to an *audit specification*. Recording those actions is done by the *target* (destination).

Audit.

هي ال **Service** الرئيسية له, حيث نقوم بتفعيلها

The SQL Server audit object is a collection of server actions or database actions (theseactions might also be grouped together). Defining an audit creates it in the off state. After it is turnedon, the

destination receives the data from the audit.

Server audit specification.

لتعفيل ال **Audit** على مستوى ال **Server**

This audit object defines the actions to collect at the instance level or database level (for all databases on the instance). You can have multiple Server Audits per instance.

Database audit specification.

لتفعيل ال **Audit** على مستوى ال **DB**

You can monitor audit events and audit action groups. Only one database audit can be created per database per audit. Server-scoped objects must not be monitored in a database audit specification.

Target.

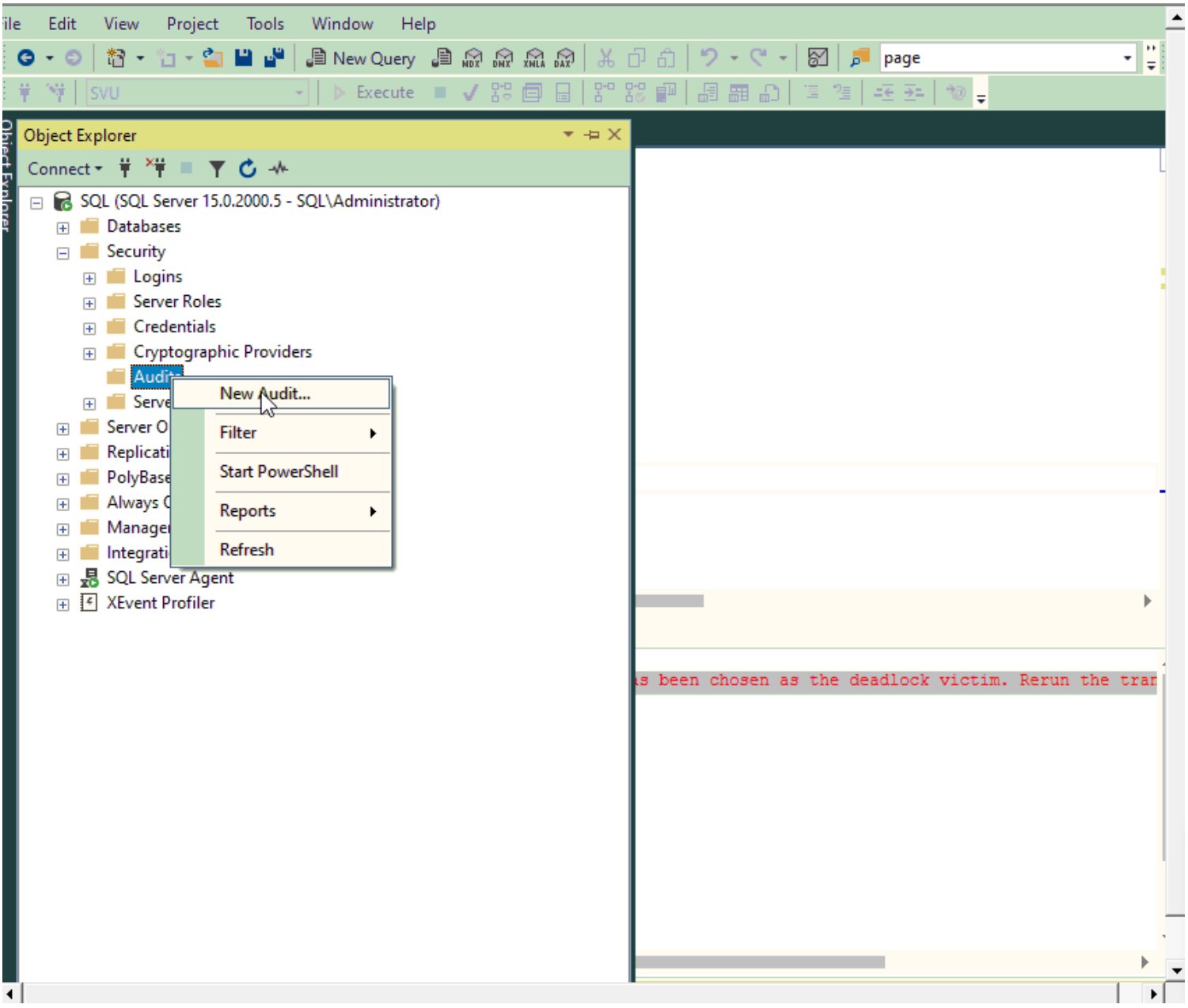
هي ال **actions** أو ال **Events** التي نريد مراقبتها, سواء من ال **DB** أو من ال **Server**

You can send audit results to the Windows Security event log, the Windows Application event log, or an audit file on the file system. You must ensure that there is always sufficient space for the target. Keep in mind that the permissions required to read the Windows Application event log

are lower than the Windows Security event log, if using the Windows Application event log. An audit specification can be created only if an audit already exists.

1.1.2. Creating a server audit in SQL Server Management Studio

- نستطيع تفعيل خدمة ال Audit بإستخدام GUI أو بإستخدام script على ال SSMS حيث هنا سنأخذ طريقة ال GUI كمثال:

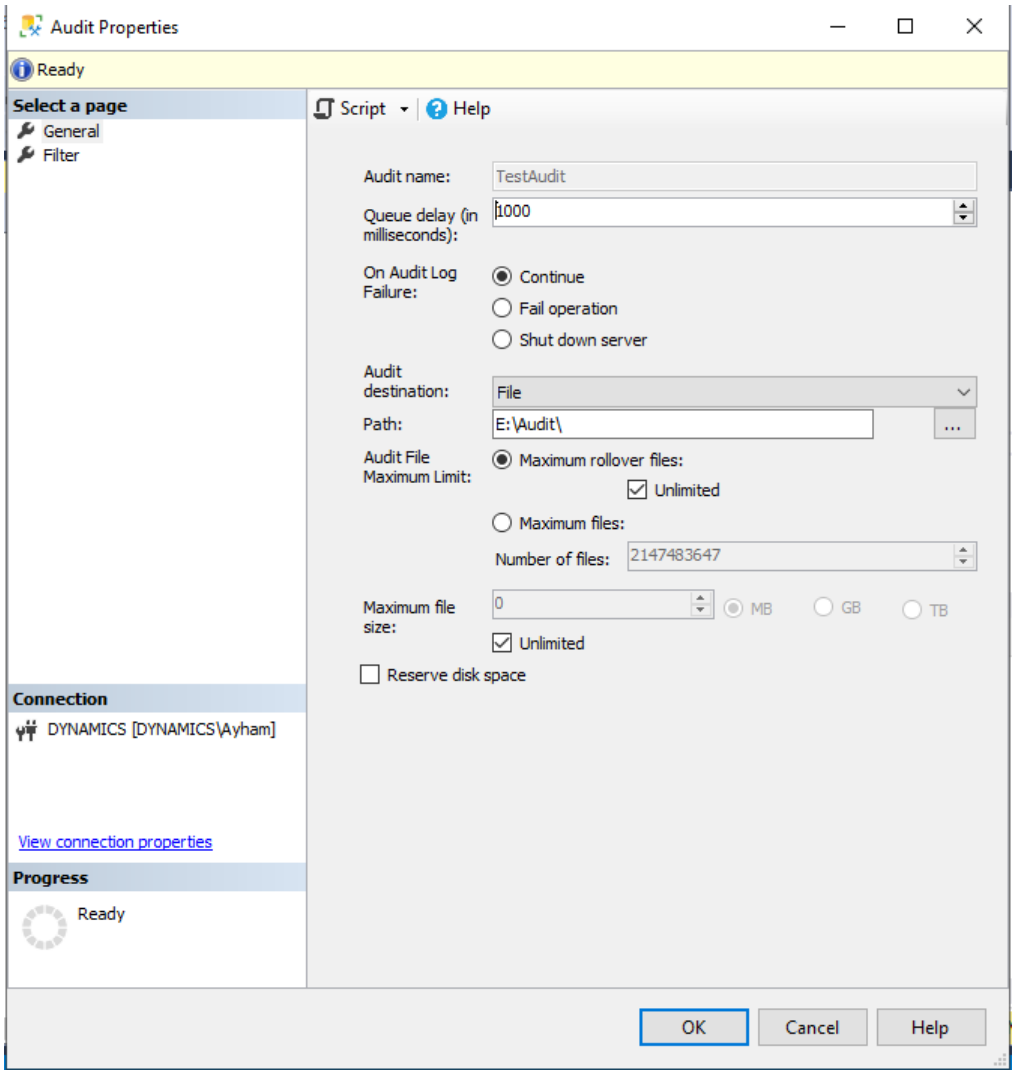


Verify that you are connected to the correct instance in SQL Server Management Studio. Then, in Object Explorer, expand the Security folder. Right-click the Audits folder, and then, on the shortcut

menu that opens, select New Audit.

In the Create Audit dialog box that opens, configure the settings to your requirements, or you can leave the defaults as is. Just be sure to enter in a valid file path if you select File in the Audit Destination list box. We also recommend that you choose an appropriate name to enter into the Audit Name box (the default name is based on the current date and time).

ثم نعطيه إسمه و المسار الذي سيقوم بتخزين الملف الخاص به فيه



ثم نقوم بالذهاب إلى ال server Audit Specification أو Database Audit Specification أو كلاهما ...

لنقوم بربطهم مع هذا ال Audit الذي أنشأناه

Remember to turn on the audit after it is created. It will appear in the Audit folder, which is within the Security folder in Object Explorer. To do so, right-click the newly created audit, and then, on the shortcut menu, click Enable Audit.

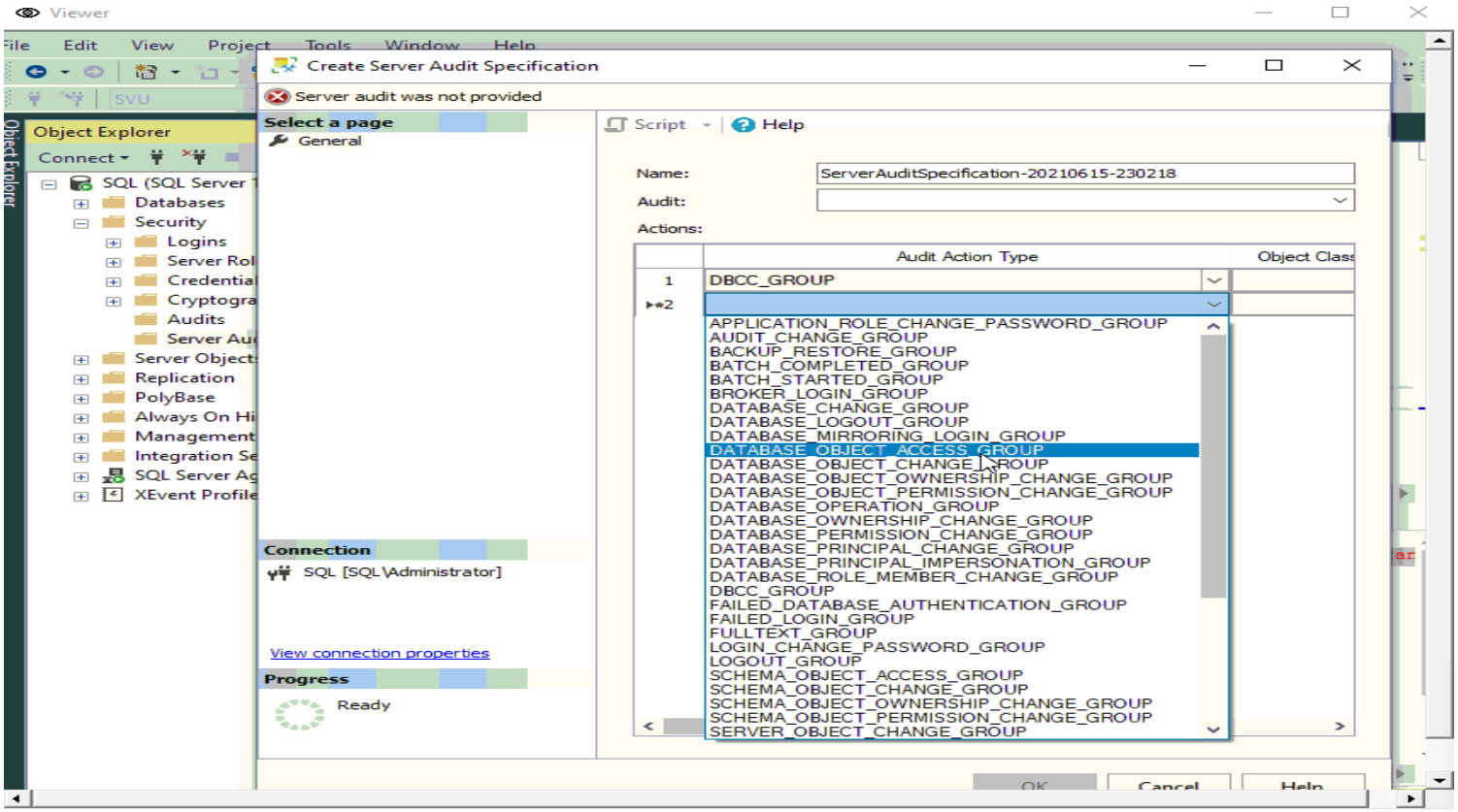
1.1.3. Create a server audit specification in SQL Server Management Studio

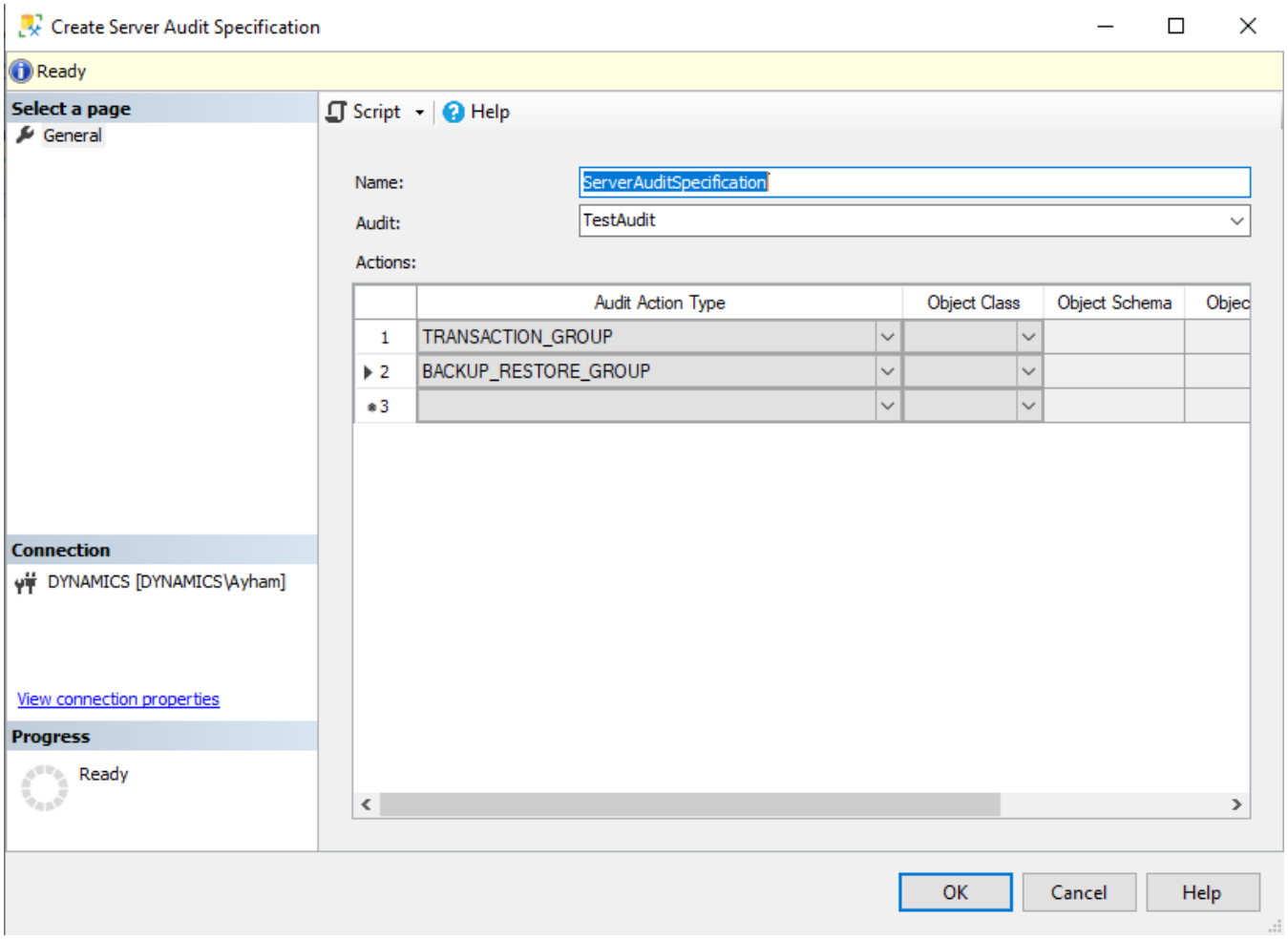
In Object Explorer, expand the Security folder. Right-click the Server Audit Specification folder, and then, on the shortcut menu, click New Server Audit Specification.

In the Create Server Audit Specification dialog box, in the Name box, type a name of your choosing for the audit specification. In the Audit list box, select the previously created server audit. If you type a different value in the Audit box, a new audit will be created by that name.

Now you can choose one or more audit actions, or audit action groups.

نقوم منها بتحديد ال Audit المراد الإرتباط معه ثم نقوم بتحديد ال Actions المراد مراقبتها:



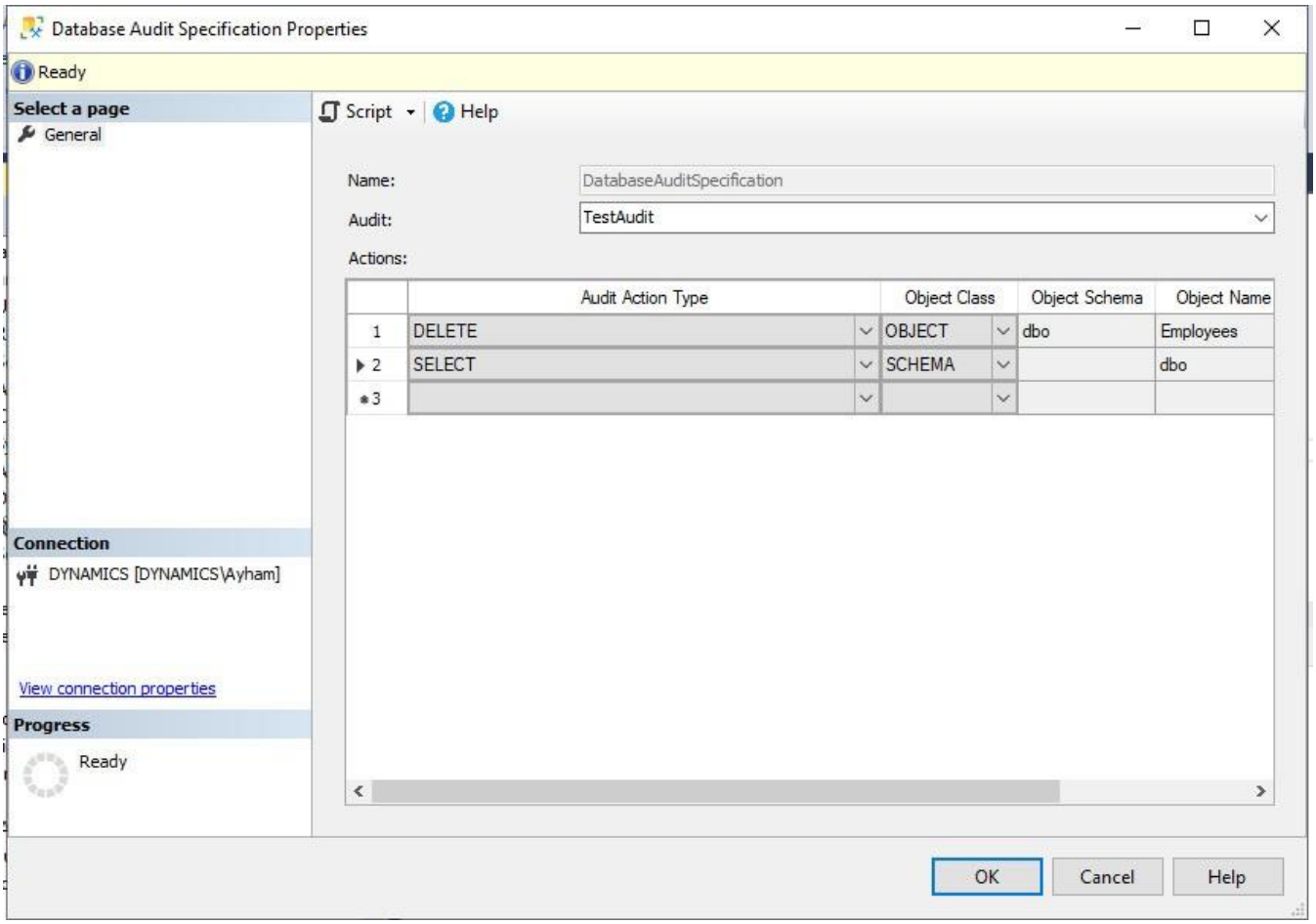

Create Server Audit Specification
Ready
Select a page
General
Script
Help
Name:
ServerAuditSpecification
Audit:
TestAudit
Actions:
Audit Action Type
Object Class
Object Schema
Objec
1
TRANSACTION_GROUP
2
BACKUP_RESTORE_GROUP
3
Connection
DYNAMICS [DYNAMICS\Ayham]
View connection properties
Progress
Ready
OK
Cancel
Help

1.1.4. Creating a database audit specification in SQL Server Management Studio

نقوم أيضا منها بتحديد ال Audit المراد الإرتباط معه, ثم تحديد ال actions المراد مراقبتها على مستوى ال DB

As you would expect, the location of the database audit specification is under the database security context.

In Object Explorer, expand the database on which you want to perform auditing, and then expand the Security folder. Right-click the Database Audit Specifications folder, and then, on the shortcut menu, click New Database Audit Specification. Remember again to use the context menu to turn it on.



1.1.5. Viewing an audit log

- يجب وجود صلاحية Control Server فما فوق لدينا لنستطيع عرض ملف ال Audit Log أو التعديل عليه

You can view audit logs either in SQL Server Management Studio or in the Security Log in the Windows Event Viewer. This section describes how to do it by using SQL Server Management Studio.

Note: To view any audit logs, you must have **CONTROL SERVER** permission.

In Object Explorer, expand the Security folder, and then expand the Audits folder. Rightclick the audit log that you want to view, and then, on the shortcut menu, select View Audit Logs. Note that the Event Time is in UTC format. This is to avoid issues regarding time zones and daylight savings.

1.2. Auditing with Azure SQL Database

- ***موجود نفس مبدأ ال Auditing في ال SQL Server لدى ال Azure SQL***

With Azure SQL Database auditing, you can track database activity and write it to an audit log in an Azure Blob storage container, in your Azure Storage account (you are charged for storage accordingly).

This helps you to remain compliant with auditing regulations as well as see anomalies give you greater insight into your Azure SQL Database environment.

You can define server-level and database-level policies. Server policies automatically cover new and existing databases.

If you turn on server auditing, that policy applies to any databases on the server. Thus, if you also turn on database auditing for a particular database, that database will be audited by both policies. You should avoid this unless retention periods are different or you want to audit for different event types.

2. Capturing modifications to data

- **لعرض التغيرات في ال Data لدينا ثلاث طرق:**

SQL Server supports several methods for capturing row data that has been modified. In this section, we discuss *Change Tracking CT, Change Data Capture CDC,* and *Temporal Tables*. Although these features allow applications to detect when data has changed, they operate very differently and serve different purposes.

2.1. Using change tracking CT

- ***يعلمنا فقط بال records التي تم تغييرها***

- يقوم ب ***Offline Synchronization*** لل ***records*** بين ال ***DB*** وال ***Data Warehouse*** مثلا أو ***DBs***

- ينصح قبل إستعماله بالقيام بعملية ***Snapshot Isolation*** لل ***DB*** لكي يتم حفظ حالة جميع ال ***records*** قبل إستخدامه

بدون تضمين ال ***records*** المتغيرة نتيجة ***Queries*** يتم تنفيذها في تلك اللحظة
- يخزن معلوماته في ال ***system tables***

Change tracking does not actually track the data that has changed, but merely that a row has changed. You use it mostly for synchronizing copies of data with occasionally offline clients or for Extract, Transform, and Load (ETL) operations. For example, an application that facilitates offline editing of data will need to perform a two-way synchronization when reconnected. One approach to implementing this requirement is to copy (a subset of) the data to the client. When the client goes

offline, the application reads and updates data using the offline copy. When the client reestablishes connectivity to the server, changes can be merged efficiently. The application is responsible for detecting and managing conflicting changes.

Configuring change tracking is a two-step process: first, change tracking must be turned on for the database. Then, change tracking must be turned on for the table(s) that you want to track. Before performing these steps, we recommend setting up snapshot isolation for the database. Snapshot isolation is not required for proper operation of change tracking, but it is very helpful for accurately querying the changes. Because data can change as you are querying it, using the snapshot isolation level and an explicit transaction, you will see consistent results until you commit the transaction.

The sample script that follows turns on snapshot isolation on the HO sample database. Then, change tracking on the HO sample database and on one tables, [Sales Header], is turned on. Only on the [Sales Header] table is column tracking activated.

```
USE master;
GO

-- Enable snapshot isolation for the database
ALTER DATABASE HO SET ALLOW_SNAPSHOT_ISOLATION ON;

-- Enable change tracking for the database
ALTER DATABASE HO
SET CHANGE_TRACKING = ON
    (CHANGE_RETENTION = 5 DAYS, AUTO_CLEANUP = ON);

USE HO;

-- Enable change tracking for Sales Header
ALTER TABLE [dbo].[Sales Header]
ENABLE CHANGE_TRACKING

-- and track which columns changed
WITH (TRACK_COLUMNS_UPDATED = ON);

-- Query the current state of change tracking in the database
SELECT *
FROM sys.change_tracking_tables;
INSERT INTO [dbo].[Sales Header] ([No],[POS Terminal NO],[Staff Id],[Date],[Time],[Shift
No],[IsCustomer],[Customer No])
     VALUES (2,"POS01",100,getdate(),getdate(),1,1,101)
Update [dbo].[Sales Header] set [Staff Id] = 100 where [No] = 1
--SET @synchronization_version = CHANGE_TRACKING_CURRENT_VERSION();
SELECT
    CT.[No], CT.SYS_CHANGE_OPERATION,
    CT.SYS_CHANGE_COLUMNS, CT.SYS_CHANGE_CONTEXT
FROM
    CHANGETABLE(CHANGES [dbo].[Sales Header],0) AS CT
```

```
INSERT INTO [dbo].[Sales Header] ([No],[POS Terminal NO],[Staff Id],[Date],[Time],[Shift No],[IsCustomer],[Customer No])
     VALUES (2,'POS01',100,getdate(),getdate(),1,1,101)

Update [dbo].[Sales Header] set [Staff Id] = 100 where [No] = 1

--SET @synchronization_version = CHANGE_TRACKING_CURRENT_VERSION();

SELECT
    CT.* FROM CHANGETABLE(CHANGES [dbo].[Sales Header],0) AS CT
```

100 %

Results | Messages

	No	SYS_CHANGE_VERSION	SYS_CHANGE_CREATION_VERSION	SYS_CHANGE_OPERATION	SYS_CHANGE_COLUMNS	SYS_CHANGE_CONTEXT	No	POS Terminal NO
1	1	3	NULL	U	0x0000000003000000	NULL	1	POS01
2	2	2	1	I	NULL	NULL	2	POS01

2.2. Using change data capture

- يقوم بنفس وظيفة ال CT بالإضافة لعرض قيمة ال Record قبل التغيير و بعد التغيير, لذلك يقوم بتخزين أكثر و بالتالي يتطلب مساحة تخزينية أعلى
- في حال وجود memory-optimized table فإن ال Change data capture يتعارض معه و لا يقوم بتسجيل التغيير في ال records الخاصة به
- لا يدعم نسخ ال Azure
- يقوم بتخزين معلوماته في Internal Tables عوضا عن ملف أو System Tables

Change data capture varies in some important ways from change tracking. Foremost, change data capture actually captures the historical values of the data. This requires a significantly higher amount of storage than change tracking. Unlike change tracking, change data capture uses an asynchronous process for writing the change data. This means that the client does not need to wait for the change data to be committed before the database returns the result of the DML operation.

NOTE

- Change data capture is not available in Azure SQL Database.
- Change data capture and memory-optimized tables are mutually exclusive. A database cannot have both at the same time.

Change data capture stores the captured changes in internal tables. The following script turns on change data capture on a fictitious database using the sys.sp_cdc_enable_db stored procedure. Then, the script turns on change data capture for the dbo.Orders table. The script assumes that a database role cdc_reader has been created

```
USE HO;
GO
EXEC sys.sp_cdc_enable_db;
```

```
EXEC sys.sp_cdc_enable_table
@source_schema = "dbo",
@source_name = "Orders",
@role_name = "cdc_reader";
```

2.3. Temporal Tables

- يطلق عليها أيضا System-Versioned Temporal Tables
- عند تفعيلها على جدول ما, يصح بإمكاننا معرفة قيمة أي حقل في تاريخ سابق حيث في حال تغيير قيمة حقل ما من هذا الجدول. فجميع القيم السابقة له لا تضيع و يمكن الإستعلام عنها.
- حيث يطلق على الجدول إسم Temporal Table الذي يحوي أعمدة إسمها Temporal يتم فيها حفظ القيمة السابقة لكل قيمة حاليا من قيم الجدول

بالإضافة لإنشاء جدول إسمه History Table يحوي كامل الحقول المعدّلة مع قيمها السابقة كافة و تاريخ كل قيمة منها مع ال Referenced schema و ال referenced table الخاصين كل من هذه القيم

-

SQL Server 2016 introduced support for temporal tables (also known as system-versioned temporal tables) as a database feature that brings built-in support for providing information about data stored in the table at any point in time rather than only the data that is correct at the current moment in time. Temporal is a database feature that was introduced in ANSI SQL 2011.

2.3.1. What is a system-versioned temporal table?

A system-versioned temporal table is a type of user table designed to keep a full history of data changes and allow easy point in time analysis. This type of temporal table is referred to as a system-versioned temporal table because the period of validity for each row is managed by the system (i.e. database engine).

Every temporal table has two explicitly defined columns, each with a **datetime2** data type. These columns are referred to as period columns. These period columns are used exclusively by the system to record period of validity for each row whenever a row is modified.

In addition to these period columns, a temporal table also contains a reference to another table with a mirrored schema. The system uses this table to automatically store the previous version of the row each time a row in the temporal table gets updated or deleted. This additional table is referred to as the history table, while the main table that stores current (actual) row versions is referred to as the current table or simply as the temporal table. During temporal table creation users can specify existing history table (must be schema compliant) or let system create default history table.

2.3.2. Why Temporal tables?

- تفيد ال **Temporal Tables** في ال:

The following are some usage scenarios of Temporal tables

1. Auditing
2. Rebuilding the data in case of inadvertent changes
3. Projecting and reporting for historical trend analysis
4. Protecting the data in case of accidental data loss

2.3.3. How does temporal work

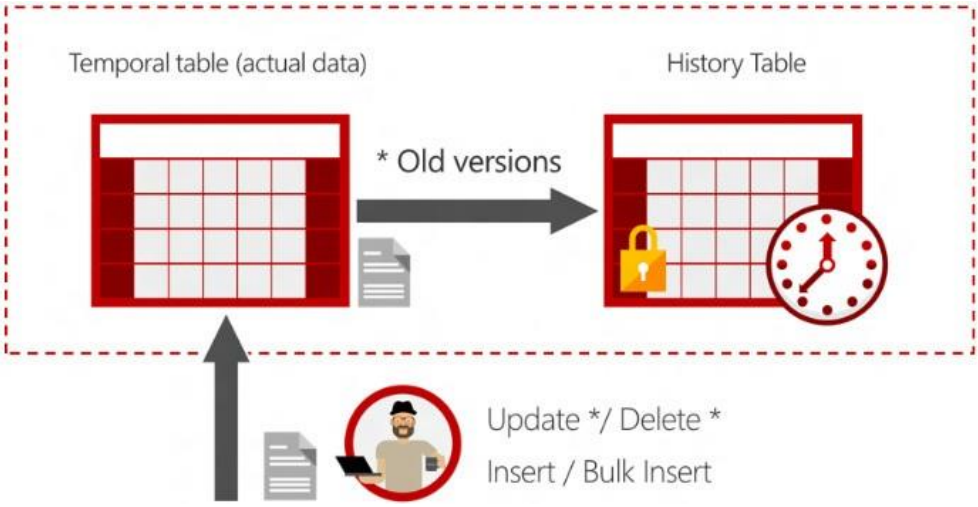
-

System-versioning for a table is implemented as a pair of tables, a current table and a history table. Within each of these tables, the following two additional datetime2 columns are used to define the period of validity for each row:

Period start column: The system records the start time for the row in this column, typically denoted as the SysStartTime column.

Period end column: The system records the end time for the row in this column, typically denoted as the SysEndTime column.

The current table contains the current value for each row. The history table contains each previous value for each row, if any, and the start time and end time for the period for which it was valid.



How do I query temporal data

The **SELECT** statement **FROM**<*table*> clause has a new clause **FOR SYSTEM_TIME** with five temporal-specific sub-clauses to query data across the current and history tables:

- As of <date_time>
- From <start_ date_time > To <end_ date_time >
- Between <start_ date_time > and <end_ date_time >
- Contained in (<start_ date_time >, <end_ date_time >)
- All

This new **SELECT** statement syntax is supported directly on a single table, propagated through multiple joins, and through views on top of multiple temporal tables.

2.3.4. Comparing change tracking, change data capture, and temporal tables

	Change tracking	Change data capture	Temporal tables
Requires schema modification	No	No	Yes
Available in Azure SQL Database	Yes	No	Yes
Edition support	Any	Enterprise only	Any
Provides historical data	No	Yes	Yes
History end-user queryable	No	Yes	Yes
Tracks DML type	Yes	Yes	No
Has autocleanup	Yes	Yes	Yes
Change indicator	LSN	LSN	datetime2

References

LeBlanc. P, Assaf. W, Jackson. B, Curnutt M; "**SQL Server 2017 Administration Inside Out**", Microsoft Press (2018).

https://docs.microsoft.com/en-us/sql/relational-databases/system-functions/changetable-transact-sql?view=sql-server-ver15

https://docs.microsoft.com/en-us/sql/relational-databases/track-changes/track-data-changes-sql-server?view=sql-server-ver15

https://docs.microsoft.com/en-us/sql/relational-databases/tables/temporal-tables?view=sql-server-ver15